THE PROBLEM BEHIND ONE-CLICK DEPLOYMENT

One repository. Thousands of deployment shapes.

DeployGuard tries to make deployment feel simple: connect a GitHub repository, let the platform understand the application, prepare what it needs, and deploy it to AWS.

THE QUESTION THAT CHANGED THE DESIGN

Before the platform can build, secure, provision or deploy anything...

How does it understand a repository it has never seen before?

WHY "DETECT THE STACK" EXPLODED IN COMPLEXITY

The combinations multiply fast.

A repository can vary across topology, framework pairing, build conventions, runtime behavior, configuration and database expectations - before developer inconsistency is added.

03 TOPOLOGY

single app / FE+BE / monorepo

08 FRAMEWORK PAIRS

React/Vite or Next x Node API/Django/Flask/FastAPI

03 BUILD PATTERNS

standard / workspace / custom

03 RUNTIME TYPES

static / Node / Python

04 CONFIG PROFILES

public / secret / optional / mixed

04 DATABASE PROFILES

none / PostgreSQL / MongoDB / external

ILLUSTRATIVE BASELINE

$$3 \times 8 \times 3 \times 3 \times 4 \times 4 = 3{,}456$$

candidate patterns in a simplified matrix - not an official support count, and not a true maximum.

Add nested directories, stale Dockerfiles, conflicting lockfiles, custom scripts, unusual names and partial apps, and the real search space becomes effectively open-ended.

THE MOMENT THE FIRST MODEL BROKE

We detected the backend correctly - and still got the app wrong.

A full-stack repository contained separate frontend and backend components. The detector found the backend, ranked it, selected it, and prepared it.

BEFORE

Repository

-> Find framework

-> Choose one app root

-> Build plan

Frontend: LOST

AFTER

Repository

-> Infer topology

-> Identify components

-> Build relationships

Frontend + Backend: KEPT

THE REALIZATION

"This repo contains Express"

is not the same as understanding the product.

WHAT THE DETECTOR HAS TO REASON ABOUT NOW

Framework detection was only signal #1.

The backend has to combine evidence from several imperfect signals, then produce a deployment plan without pretending the repository is cleaner than it is.

01 **STRUCTURE**
app roots, nesting, workspaces

02 **MANIFESTS**
package.json, Python manifests, lockfiles

03 **FRAMEWORK**
frontend, backend, SSR, static behavior

04 **COMMANDS**
build, start, custom scripts

05 **CONFIG**
public, secret, required, optional

06 **RELATIONSHIPS**
frontend <-> backend <-> database

07 **RUNTIME**
static, Node, Python, health behavior

08 **CONTAINERS**
existing Dockerfiles vs generated plan

Goal: **build a component-aware plan for the complete application, not just a confident guess about the first framework we found.**

THIS LIVES IN THE BACKEND, NOT ONLY IN A DIAGRAM

The detector became a real inference layer.

The implementation scans roots and manifests, builds candidates, ranks plausible applications, discovers units and infers relationships before producing a deployment profile.

```
const SUPPORTED_FRAMEWORKS = new Set([
  "streamlit",
]);

You, 2 days ago | 1 author (You)
@Injectable()
export class StackDetectionService {
  private readonly logger = new Logger(StackDetectionService.name);
  private readonly unitDiscovery = new ApplicationUnitDiscoveryService();
  private readonly relationshipInference = new RelationshipInferenceService();

  constructor(
    private readonly templateMatchingService: TemplateMatchingService,
    private readonly deployabilityScanner: RepoDeployabilityScannerService,
    private readonly detectorResolver: MainstreamDetectorResolverService = new MainstreamDetectorResolverService()
  ) {}

  detect(
    workspacePath: string,
    commitSha: string | null,
    preferredAppDirectory?: string | null,
    overrides: ProjectDeploymentOverrides = {},
    includeTopology = true,
  ): DeploymentProfileDraft {
    const scanRoot = this.resolveScanRoot(workspacePath, preferredAppDirectory);
    const preferredDirectory = relative(workspacePath, scanRoot).replace(/\\/g, "/") || ".";
    const manifestFiles = this.findManifestFiles(scanRoot);
    const candidates = this.buildCandidates(scanRoot, manifestFiles);
    const selected = this.selectCandidate(candidates);
    const rankedCandidates = candidates.map((candidate) => ({ directory: candidate.relativeDirectory, score: this.
    const plausibleCandidates = rankedCandidates.filter((candidate) => candidate.score >= 100);
    const appRootAmbiguous = !preferredAppDirectory && plausibleCandidates.length > 1 && plausibleCandidates[0].sc
```

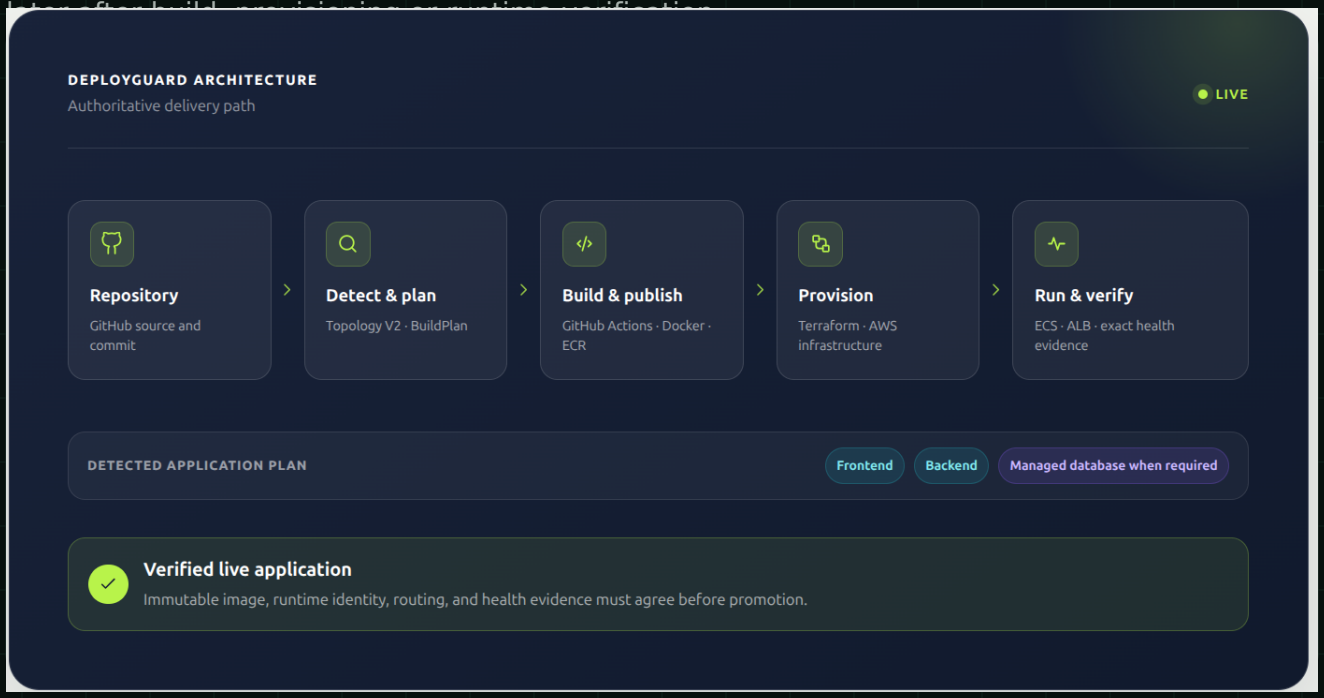
SCAN ROOTS · BUILD CANDIDATES · RANK · INFER RELATIONSHIPS

The important part is not the class name. It is the shift from one-shot framework matching to evidence-driven repository interpretation.

WHY SMALL DETECTION MISTAKES BECOME BIG DEPLOYMENT FAILURES

One bad assumption can survive half the pipeline.

The dangerous part: a wrong decision may look valid during detection and only fail minutes later after build, provisioning or runtime verification.



- **JS FRONTEND** may become static nginx, not a Node server
- **VITE_*** may be public config without being a URL or required
- **HEALTH ROUTE** backend can be healthy while the platform checks the wrong path

WHAT THIS FYP BECAME FOR US

We did not choose the easiest project to finish.

We chose a real infrastructure problem that could force us to learn across application structure, automation, cloud provisioning, runtime behavior and failure - and turn that learning into something worth building.

DEPLOYGUARD / TEAM

Hassan Sajjad
Faria Fatima
Tania Khawar

Built together. Special appreciation to Faria and Tania for stepping into cloud infrastructure and unfamiliar system problems without being afraid of the complexity.

THE LINE WE KEEP COMING BACK TO

The developer should only have to say:

"Deploy this."

Our job is understanding what "this" means.

The simpler deployment feels on the surface, the more uncertainty the platform has to absorb underneath.